Crickbot workshop: speaker script

About 53 minutes of talk plus Q&A. For a 30-minute slot, skip slides 13, 16, 17, 22 and 24, and shorten the demo to slides 29 and 31.

Before you start (15 minutes early)

1. `cd ~/being-session-3.0-architect/cricket-chatbot-app && scripts/start.sh` (restarts the match at ball 1)
2. Open http://127.0.0.1:8000 and http://127.0.0.1:8000/#ops in two tabs; unlock the dashboard with the admin token
3. Ask one AI question to warm the model (for example: Why is Malinga so effective at the death?)
4. Open both databases in DB Browser and a terminal in the project folder
5. Close heavy apps; set the screen to not sleep; zoom the browser to 125% so the back row can read it

Rules

Say:

- Numbers you measured, with the word 'measured'
- The weaknesses, before anyone asks (slides 5 and 32)
- '10 years of IPL season data, 2008 to 2017'
- One key line per section, then move on

Avoid:

- 'It's production ready' or 'it's fully secure'
- 'It can handle 5 million users'
- Reading slides word for word
- Untested questions in the live demo
- Apologising for it being a laptop demo

Slide by slide

1. Title · 1 min

Say: Good morning everyone. In part 1 we designed a chatbot for 5 million cricket

fans during an India–Pakistan final. Today I'll show you that I actually built it, as a working version on my laptop. I'll walk you through how it's designed, how the data is stored, what I decided and why. And then I'll break it live in front of you.

Do: Have the app running and the dashboard open in a second browser tab before you start. Pause for a second after 'break it live'.

Avoid: Don't apologise for it being 'just a laptop demo'. Don't start with technology names.

2. What I built, in numbers · 1.5 min

Say: Four numbers. 150,460: that's every ball of 10 IPL seasons, 2008 to 2017. 827 documents in the vector database. 153 automated tests. And the one I'm proudest of: about 150 milliseconds for 9 out of 10 answers, because most questions never touch the AI model. Everything runs locally: React, FastAPI, Ollama, Chroma and SQLite. No cloud, no API keys.

Do: Point at each number as you say it.

Avoid: Don't explain each technology yet; that comes in the architecture part.

3. Agenda · 0.5 min

Say: Six parts: the problem, the architecture, the data, my decisions, quality, and then the live demo. Please keep questions for the end; I've kept time for them.

Avoid: Don't read out all six descriptions.

4. The brief, and what I built to show it · 1.5 min

Say: On the left is the brief from part 1. On the right is what I built to prove it. I don't have India–Pakistan data, so I used 10 years of real IPL data, and I replay the 2017 final ball by ball, one ball every 4 seconds. Mumbai won that final by one run. The scale is a laptop, but every design idea is real: fast paths, guardrails, rate limits, load shedding and monitoring.

Avoid: Don't claim it handles 5 million fans. Say clearly: the laptop proves the design, not the scale.

5. Requirements, and how the build measures up · 1.5 min

Say: Functional requirements on the left are all done. On the right are the targets from part 1 against what I measured. Green ticks for data-answer speed, correctness, partial outages, overload and security. Two red crosses, and I want to be upfront about them: AI answers are slow on a CPU, and the app is a single process, so it can't scale out yet. Both are on the roadmap.

Do: Point at the two red crosses yourself, before anyone else does. It builds trust.

6. C4 level 1: system context · 1.5 min

Say: C4 is a way to draw architecture at different zoom levels. I'll show three. Level 1 is the big picture: who uses it, and what it depends on. Two people: a fan who chats, and an operator who runs it. Three outside things: the ball-by-ball feed, the Ollama runtime that hosts the AI models, and the IPL data files. Dashed boxes are outside the system. Nothing leaves the laptop.

Avoid: Don't go into protocols or ports here.

7. C4 level 2: containers · 2 min

Say: Zoom in one level: the containers, meaning things that run or store data. The browser app talks to one FastAPI process. Inside it: the HTTP layer with security and rate limits, the chat agent in red, the live feed, the monitor and the health registry. On the right are three stores. Ollama runs the models, SQLite holds the exact stats, and Chroma holds the searchable knowledge. The ingest job at the bottom runs once and turns the CSV files into both databases. Two databases on purpose: SQL for exact numbers, vectors for meaning.

Do: Key line to land: 'SQL for exact numbers, vectors for meaning.'

8. C4 level 3: inside the chat agent · 2.5 min

Say: This is the heart of the system: 13 steps every question goes through. Read it like a snake: left to right, then back, then forward again. The green boxes can answer and stop: instant answers for the score, a cache, scope checks, and the fact composer that writes answers straight from the database. Only questions that need reasoning reach step 10, the AI model, in red. After the model come two safety checks: every number must exist in the data, and the model can never repeat its own instructions, even while it's streaming.

Do: Explain the colours first, then the red box, then steps 11 and 12.

Avoid: Don't walk through all 13 boxes one by one; you'll lose the room.

9. One question, end to end · 1.5 min

Say: This is the same pipeline drawn as a funnel. Each bar is a place where a question can be answered and leave. The cheap checks come first: zero milliseconds. Facts from the database take about 150 milliseconds. Only the bottom bar is slow: the AI model, 7 to 17 seconds on this CPU. In my 22-question accuracy check, 20 never reached the model. That is the single biggest speed decision in the project. The fastest AI call is the one you don't make.

Do: Slow down on the last sentence; it's the line people remember.

10. The live match · 1 min

Say: The live feed stands in for a real data provider. One writer owns the match state. Every ball increases a version number, and duplicates are ignored. Browsers get updates over server-sent events. Every answer shows which ball it was based on, so a fan knows how fresh it is.

Avoid: Don't go deep into SSE versus WebSockets unless someone asks; it's in ADR-07.

11. 10 years of IPL data in three SQLite tables · 2 min

Say: Now the data. 10 IPL seasons, 2008 to 2017: 636 matches, 150,460 balls, 495 players. Two CSV files become three SQLite tables. deliveries is the big one: one row per ball. One important point: the database is opened read-only, and the AI never writes SQL. It can only call seven fixed queries. So even a successful prompt injection can't touch the data.

Do: This is where you say '10 years of season data' clearly.

12. Chunking: one document per fact · 2 min

Say: Everyone asks how I chunked 10 years of data. I didn't cut text into 1,000-character pieces. Each document is one complete thing: a match, a player, a season, a team or a rule. That gives 827 documents. Why? If you cut every 1,000 characters, Kohli's 4,423 runs could land in one chunk and his strike rate in the next. The 150,000 individual balls stay in SQLite, because vector search is the wrong tool for exact numbers. Each document becomes 768 numbers using the nomic embedding model. Chunk by meaning, not by size.

Do: Key line: 'Chunk by meaning, not by size.'

13. Where everything is stored on disk · 1 min

Say: Everything is plain files. The CSVs are the source of truth. backend/var holds the

23 MB stats database and the Chroma folder: text and metadata in a SQLite file, and the fast search index in binary files. I'll open both databases in the demo.

Do: Skip this slide if you're short on time.

14. Hybrid retrieval · 2 min

Say: Plain vector search had a real failure. 'Which team won the 2016 final' returned five random 2016 league matches, because 60 matches from one season look almost identical as vectors. So I combined four signals: a season filter, vector similarity, keyword scoring so the word 'final' counts, and a question-type signal. Then I fused the rankings. I measured it: the right document comes first on all 38 test questions, including 14 fresh ones I never tuned against.

Avoid: Don't say '100% means perfect'. Say 'on 38 test questions'.

15. Architecture decision records (1 of 3) · 1.5 min

Say: Architecture decision records: what I decided, why, and what it cost. I'll pick the important ones. ADR-02: facts come from data, words from the model. ADR-03: answer without the AI whenever possible. These two decisions are why it's both fast and accurate.

Avoid: Don't read all four cards. Pick two; the rest are there to read later.

16. Architecture decision records (2 of 3) · 1 min

Say: ADR-08 is about protecting a slow CPU model: one answer at a time, a short queue, and anyone beyond that gets a quick data answer instead of a timeout.

Avoid: Don't read every card.

17. Architecture decision records (3 of 3) · 1 min

Say: ADR-10 is the honest one: state lives in memory, so I can't run two copies yet. That's the first thing to move to Redis. Every decision has a trade-off, and that's the point of writing them down.

18. Security and guardrails · 2 min

Say: Security is built in seven layers, so no single check has to be perfect. One real story: before I hardened it, typing 'ignore previous instructions' with numbers in place of letters made the model print its own instructions word for word. Now the input is cleaned before it's checked, and every answer is compared with the system prompt while it streams. I attacked it with 13 prompts and all were blocked. Then I threw about 110 strange inputs at it, which found 15 bugs, all fixed.

Avoid: Never say 'it's completely secure'. Say 'defence in depth; pattern lists can still miss new phrasings'.

19. When something breaks, the rest keeps working · 1.5 min

Say: Every dependency has its own circuit breaker, so failures stay partial. If the AI model dies, fans still get the live score, stats and rules. If the vector database dies, stats still work. Only the last red row isn't handled: if the whole process crashes, nothing restarts it yet. I'll show the first two live in the demo.

20. Rate limiting, throttling and load shedding · 1.5 min

Say: Three different protections. Rate limiting is per person: one fan can't flood it, at 20 questions a minute with a burst of 6. Throttling is global: total capacity, 10 requests a second for everyone together. Load shedding protects the slow model: one answer running, six waiting, and everyone beyond that gets a data answer in about a second instead of a minute-long timeout. All three are one-click tests on the dashboard.

Do: Be ready for 'what's the difference between rate limiting and throttling?' Answer: per person versus total.

21. Latency: where the time goes · 1.5 min

Say: This chart uses a log scale, because the spread is huge: one millisecond for the score, about 150 for database answers, about 12 seconds for the model on this CPU. 99% of that is the model itself: it reads at 77 tokens a second and writes at 11. The honest production target isn't a full answer in 500 milliseconds. It's the first word in under 500 milliseconds on a GPU, then streaming the rest.

22. Capacity · 1 min

Say: Measured on this laptop: about 385 requests a second for fast answers, but only one AI answer at a time. For 5 million fans, the part-1 design applies: a CDN for the scoreboard, Redis, many app copies and GPUs.

Avoid: Don't multiply laptop numbers to claim production capacity.

23. Observability: the dashboard · 1 min

Say: Observability is built in; there's no Grafana needed. There's a status banner on top, 12 live charts, a Break button for every component, and the traffic test lab. You'll see it live in a few minutes.

24. Where to look when something breaks · 1 min

Say: When something breaks, this is the order I look in: the dashboard to see what's down, the trace to see which step was slow, and then the log line, found by request id. Every response carries that id.

Do: Skip this slide if you're short on time.

25. Testing · 1.5 min

Say: Six kinds of testing. 153 automated tests run the whole app with a fake model in 5 seconds. Then the real-model checks: 22 out of 22 answers correct against facts read straight from the database, 10 out of 10 failure drills, and all three traffic tests pass. And fuzzing found 15 real bugs; each one now has a test so it can't come back. I didn't only test that it works. I tested how it breaks.

Do: Key line: 'I tested how it breaks.'

26. Demo: run it in six steps · 1 min

Say: If you want to run it yourselves: start Ollama, run setup once, then start.sh, and open the browser. That's it. Now let me show you.

Do: Switch to the browser here.

Avoid: Don't run setup live; it takes minutes.

27. Demo: open the databases · 2 min

Say: This is the real data behind every answer.

Do: Stats DB window: Browse Data → deliveries (150,460 rows). Execute SQL → the top run scorers query → SK Raina 4548 comes first. Chroma window: embeddings (827 rows), then embedding_metadata (show the text). Terminal: python3 scripts/search.py "Who won the 2016 final?" and point at the distances.

Avoid: Don't edit anything in DB Browser; it's opened read-only anyway.

28. Demo: what to point at in the chat · 0.5 min

Say: On every answer, look at four things: which lookups ran, where the answer came from, which ball it's based on, and the sources.

29. Demo script: ten questions that work · 4 min

Say: Mostly let the app talk. For the last one: 'This one needs reasoning, so it goes to the model. While it's thinking, notice the answer will show figures checked.'

Do: Type questions 1 to 9 quickly and point at the badge and the time each time. Do the Hindi and Hinglish ones; they always get a reaction. Ask the Malinga question last and keep talking for the 12 seconds it takes.

Avoid: Don't type other questions while the AI one is running. Don't improvise new questions in front of the audience unless you've tried them before.

30. Demo script: five inputs it must refuse · 2 min

Say: Notice that every refusal is instant; no AI is involved.

Do: Type the five inputs in order. On the first one, tell the story: before the fix, it leaked the system prompt.

31. Demo: break things on purpose · 3 min

Say: This is the most important minute of the demo: the bot keeps working while a part is down.

Do: Dashboard → Break Chat model → the banner turns red → go to the Match tab → ask 'Who won IPL 2012?' → still answered → back to Dashboard → Restore. Then Test lab → run the three tests → three PASS. Finish on Alert history.

Avoid: Don't forget to press Restore. Check the dashboard is green before you move on.

32. Where it can still be wrong · 1.5 min

Say: Before you ask, here's where it still gets things wrong. My favourite: it said Kohli's 129.9 strike rate beats Rohit's 130.9. The numbers are right, but the logic is backwards. My checker verifies numbers, not reasoning. That's a 3-billion-parameter model on a CPU.

Do: Say this confidently. Knowing your weaknesses is a strength.

33. What's covered, and what's pending · 1 min

Say: On the left: everything working and shown today. On the right: what a real launch still needs. None of the right side is a redesign; they're known, standard pieces.

34. Road to production · 1 min

Say: P0, before real users: HTTPS and login, Redis, auto-restart, a GPU and CI. P1 is about scaling out to the part-1 design. P2 is about staying correct over time.

35. Three takeaways · 1 min

Say: Three things to remember. Facts from data, words from the model. Make the slow path rare. And assume it will break, and build it so you can see it and survive it. Thank you. Questions?

Do: Stop talking after 'Questions?' and leave the slide up. Use the Q&A cheat sheet in the printed script.

Q&A cheat sheet

Why not ChatGPT or a cloud model? No cost per question, data stays on the machine, and it works offline. The model is a setting: switching to a GPU server or a hosted model doesn't change the design.

Why two databases? SQLite for exact numbers (runs, wickets, results); Chroma for meaning (rules, descriptions). Vector search is bad at exact figures.

How accurate is it? 22 of 22 on the answer check and 38 of 38 on the search check. Those are small regression sets, not a guarantee; the next step is evals from real fan questions.

Does it hallucinate? It can. Numbers not in the data are removed, out-of-scope questions get an honest reply, and 9 in 10 answers don't use the model at all. Reasoning can still be wrong (slide 32).

How would it scale to 5 million fans? Stateless app copies behind a load balancer, Redis for shared state, a CDN for the scoreboard, a GPU fleet or hosted model, and answers pre-computed once per ball.

Rate limiting vs throttling? Rate limiting is per fan or IP. Throttling is the global total. Load shedding decides what to do with work beyond capacity.

Why SSE, not WebSockets? The scoreboard only needs server-to-browser updates. SSE is simpler, works through proxies, and reconnects automatically.

Is it production ready? No, and here's exactly why: slides 33 and 34. The P0 list is what I'd do first.

What if the feed sends a wrong ball? Each ball has a sequence number; duplicates are ignored and a correction becomes a new version, which retires cached answers.

How long did it take? Answer honestly, and mention it was built with an AI coding assistant if that's the case. That's a good discussion point for an architecture workshop.

If something goes wrong during the demo

- **An AI answer is very slow:** say "this is the CPU path, exactly why 9 in 10 answers avoid it", then ask a stats question to show the fast path.
- **You get a 'slow down' message:** that's the rate limit working. Say so, then wait 10 seconds.
- **The app doesn't respond:** run `scripts/start.sh` again. Meanwhile, show slides 23 and 28 (screenshots of the real app).
- **The match has ended:** Dashboard → Operator → Restart match.